

Generational Garbage Collection for Lazy Graph Reduction

Julian Seward*

sewardj@uk.ac.man.cs

Abstract

Although the LISP community have been exploiting the fruits of generational garbage collection for some time, little attempt has been made to apply these ideas in lazy functional language implementations. This paper attempts to plug that gap.

The action of overwriting an unevaluated thunk with its final value, known as *updating*, is central to lazy reduction systems. Unfortunately, updating creates pointers from older to younger generations. A simple two-generation scheme which allows heap occupancy to approach 100% is presented. This collector is a hybrid semispace and mark-scan collector. We show that keeping track of old-to-new pointers imposes virtually zero time and space overhead on the mutator. Consequently a net performance gain can be had by using generational collection.

This paper describes how a generational collector was incorporated into a standard G-machine interpreter. Detailed performance measurements presented indicate that a significant improvement in overall performance is achieved, compared to both semispace and compacting mark-scan collectors. Some interesting variants of the basic scheme are discussed. Finally, a possible compiled-code implementation is presented.

Keywords

Garbage collection, Generational, Graph reduction, Lazy, Functional, Updating.

1 Introduction

1.1 The problem

Recent work indicates there is much to be gained by employing a garbage collection strategy which exploits cell lifetimes. Quite a few generational schemes have been suggested and implemented with considerable success. A recent example is Standard ML of New Jersey [App92].

* Author's address: Department of Computer Science, Victoria University of Manchester, Oxford Road, Manchester M13 9PL, UK. Tel: +44 61 275 6291 Fax: +44 61 275 6236

Observations show that if a cell survives one garbage collection, it is likely to survive several more. The central principle of any generational collector is to segregate cells which look like they will last a long time, and collect them much less frequently than the rest. When heap occupancy is high, this could be a big win over non-generational schemes.

A key problem is how to deal with pointers from older to newer generations. Such pointers are created in a lazy reduction system when unevaluated expressions, or *thunks*, are overwritten with their final value. This action is known as *updating*. With strict languages like SML and LISP, updating is unnecessary, so old-to-new pointers only appear if assignment is used. This may explain the lack of takeup of generational ideas in the lazy arena.

This paper describes how a generational collector was incorporated into a standard G-machine interpreter [Joh87]. Detailed performance measurements indicate that a significant improvement in overall performance is achieved, compared to both semispace and compacting mark-scan collectors. The mutator is not significantly impeded by the need to detect old-to-new pointers, and heap utilisation may approach 100%.

1.2 Structure of paper

Three main sections discuss theory, results and further work:

- **Section 2** derives a suitable generational collector by merging two well-known non-generational collectors. Next, relevant details of the G-machine implementation in question are examined. A strategy for dealing with updates is defined, and we consider under what circumstances this will work well.
- **Section 3** presents detailed performance results for the generational collector. We also compare its performance to the same G-machine using a semispace collector, and a compacting mark-scan collector. This provides an illuminating insight into the relative strengths and weaknesses of the new collector.
- **Section 4** introduces some optimisations which have not yet been implemented. It also discusses how this collector might be integrated into a compiled-code reduction system which employs “info-table” style cell tags, as implemented in the STG machine [Pey91] and the Chalmers G-machine [Joh87].

2 A generational collector

2.1 A starting point: the semispace collector

Two important properties that a good sequential garbage collector should possess are:

- **Compaction.** It is widely accepted that allocating from a contiguous block is essential for good mutator performance.
- **Efficiency.** Functional language implementations place tremendous demands on their collectors. For example, the Chalmers LML compiler [Aug84] frequently achieves an allocation rate well in excess of a megabyte per second on widely

available workstations. If we demand that garbage collection takes at most 20% of execution time, we imply a *minimum* recovery rate of five times the allocation rate. A few back-of-the-envelope calculations reveal that the recovery rate required begins to approach the instruction rate of the processor. Clearly, efficiency is paramount.

Possibly the most promising candidate is the semispace collector [Che70]. It is very simple to implement. An absolutely crucial property is that collection time depends only on the number of *live* cells in the heap. Consequently we can achieve a recovery rate asymptotically approaching infinity simply by making the heap arbitrarily large¹.

The high recovery rate of Chalmers LML mentioned above is attained by employing just such a collector. Unfortunately, the semispace scheme has three flaws, all of which we now attempt to correct.

1. **Heap occupancy is limited to 50%.** Not only does this cause a serious under-utilisation of a valuable resource, it is also extremely annoying to find that one's program has run out of space when there are megabytes of memory which could be used if only the collector made better use of available resources.

Proponents of semispace collection have in the past claimed that a virtual memory system alleviates the problem since "the unused semispace is simply paged out, freeing up real memory for the current semispace". Recently, a few voices of dissent have pointed out that this causes large amounts of paging in practice. The author would like to go further and point out that argument is absolutely invalid. For the argument to stand would require disk I/O transfers to operate sustainedly at memory speeds.

In any case, not everyone has a virtual memory machine.

2. **Old cells are copied repeatedly.** This problem is shared with all non-generational compacting schemes. It seems a pity to waste time indiscriminately moving old cells again and again given that we can identify the majority of them at very little cost.
3. **Locality is appalling.** The mutator cyclically visits every cell in the heap before returning to the start. This constitutes worst-case behaviour from the viewpoint of both the cache and the virtual memory system.

This all looks like bad news. Let us restate the advantages of semispace collection:

1. **Speed.** In a sparsely occupied heap the semispace method outperforms all others by a considerable margin.
2. **Simplicity.** Ease of implementation is important.
3. **Space.** No auxiliary data structures are required.

If it were possible to use this collector as the basis of a hybrid system, we might be on to a good thing. It is important to get heap utilisation as high as possible, so we next look at a second collector.

¹ In practice, collection time is also proportional to the number of roots, but this effects all implementations equally.

2.2 The compacting mark-scan collector

Such a collector has the advantage of allowing heap utilisation up to 100%. Operation is three-phase, as follows:

1. **Mark the accessible graph.** This involves a recursive traversal, starting from all root pointers.
2. **Compact.** All live cells are slid to one end of the heap, leaving a contiguous free block.
3. **Fix up pointers.** Since all live cells have moved, it is necessary to adjust pointers to cells so as to reflect their new locations. Root pointers are similarly adjusted.

This is expensive. Phase (2) involves a complete scan of the heap even if occupancy is low. So simply adding an arbitrary amount of memory does not necessarily increase collector efficiency in this case. In practice, measurements show that at low occupancy, this collector performs badly compared to the semispace collector. When occupancy gets higher, though, they are more evenly matched.

A naïve implementation requires an auxiliary stack to guide the mark phase. In the worst case, this can be as big as the heap itself. We also need a table in which to record the new locations of cells after phase (2), although in practice it is possible to re-use the mark-stack for this.

At the cost of considerable extra complication, both the mark and fixup phases can be done in constant space. Marking using pointer reversal alleviates the need for a mark stack [Pey87]. Using Jonker's in-place compaction algorithm [Jon79], references to cells are chained together, so the new-address table is eliminated.

2.3 Merging the two

The following combination, suggested in [San91], is a variant of the generational collector employed in SML-NJ [App92]. Appel's collector, like this one, divides the heap into two generations. However, both generations are collected by copying, so occupancy cannot exceed 50%, a serious limitation. We employ the compacting mark-scan scheme to collect the older generation, and thereby allow occupancy arbitrarily close to 100%.

As depicted in Figure 1, the heap is divided into three regions, **OldSpace**, **ToSpace** and **FromSpace**. The latter two are equally sized. Old cells are kept in the section delimited by **HeapStart** and **OldEnd**. New cells are allocated in **FromSpace**, moving towards **HeapEnd**. Eventually **FromSpace** becomes full. A copying collection then moves all live cells in **FromSpace** to **ToSpace**, adding them to the end of **OldSpace**. **OldEnd** is moved along to reflect this, so the cells collected enter the **OldSpace**. Such an event is called a *minor collection*. The remaining space is split again and allocation in the now diminished **FromSpace** resumes.

After some number of minor collections have gone by, **OldEnd** will have advanced past **OldMax**. We then perform a compacting mark-scan collection of the entire heap, that is to say, of **OldSpace**, since a minor collection has just been performed. Hopefully, this *major collection* causes **OldEnd** to retreat considerably towards **HeapStart**, in

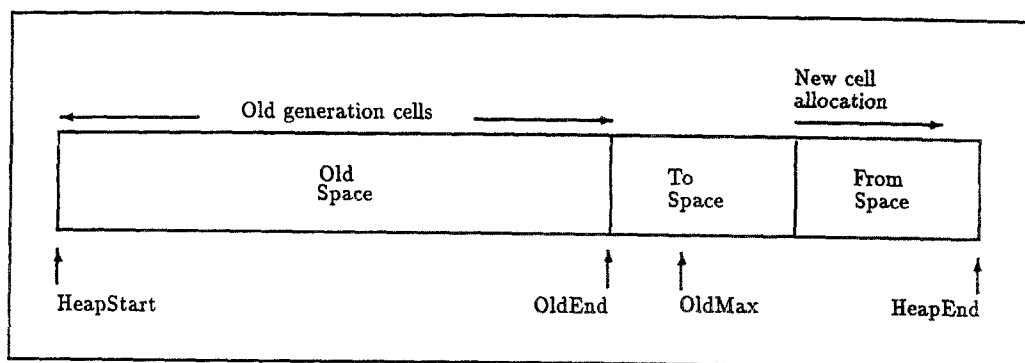


Figure 1: Heap organisation for generational collection

which case we proceed as normal. But if **OldEnd** still exceeds **OldMax**, it is necessary to deem the heap full and abandon execution. Clearly, then, maximum allowable occupancy is defined by **OldMax**. This setting also has a bearing on overall efficiency. Experimentation shows a value of 90% gives acceptable performance.

Alas, there is a problem. When a cell in **OldSpace** is updated, it may acquire pointers to cells in **FromSpace**. During a minor collection, **FromSpace** cells are moved, invalidating such pointers. So it is necessary to keep a record of all such updates, and fix up the **OldSpace** cells after each minor collection. But more than that, such cells must act *as a source of roots* during the minor collection, for it is conceivable that a **FromSpace** cell is referred to only from **OldSpace**.

So we need an auxiliary array of pointers to old-space cells which have been updated. Henceforth, this array is referred to as the *forward reference table*. Supposing a cell at location x is updated. Under what conditions is it necessary to put x into the table?

- x must be in **OldSpace**: $x \leq \text{OldEnd}$.
- The new contents of x must contain one or more pointers.
- At least one of these pointers should point to **FromSpace**.

The second and third tests are optional. They reduce the required forward reference table size somewhat, but not a lot. When a minor collection is performed, the table acts as a source of roots.

The viability of our scheme depends on

- The required size of the forward reference table.
- How much of an overhead the update tests impose on the mutator.

In the next section we present figures which indicate that for typical programs, at least 97% of all updates are in the new generation. Consequently, if **FromSpace** contains, say, 300000 closures, approximately 10000 words of forward reference table are needed. In the STG machine, all closures contain at least 2 words, from which it is possible to conclude

that the table size is only about 1.5% of overall heap size. Old-space updates occur when there is a significant delay between the creation of a thunk and its updating, because in the delay, the root cell may well be transported into the old generation. Most programs update the majority of their thunks quickly, and thereby cause few old generation updates. However, a degenerate case that has up to 20% of its updates in the old generation has been discovered. This is somewhat alarming, but hopefully it does not occur very often.

We need, therefore, a way to deal with the rare situation where the forward reference table becomes full. In this case the minor collection can be started early, at the cost of a little extra inefficiency.

2.4 Details of implementation

The collector was incorporated into a G-machine interpreter written in Sun Pascal. The system is based closely on the supercombinator language presented in [Pey87]. Importantly, the only data structures supported are lists. Programming in the language is a bit like working with a lazy variant of LISP. All heap cells are the same size, with the following layout:

- Tag - 1 byte
- Mark bit - 1 byte
- Left pointer - 4 bytes
- Right pointer - 4 bytes

The equal sizedness of all cells has an important bearing on this paper: all updates are done by copying. There are *no indirection nodes*. Despite this, most of the work described here is also relevant to systems using indirection nodes.

The use of an interpreter to gather timing information is also of some concern. Whether or not the relative timings presented below accurately reflect what would happen in a compiled-code implementation is a matter for debate. Nevertheless, our interpreter does make significant demands on the heap. Running on a Sun-4/330, it averages around 200000 G-codes per second with heap allocation varying from 45000 to 60000 cells per second. This is roughly equivalent to half a megabyte per second, a third of the observed allocation rate of Chalmers LML on the same machine. It seems therefore reasonable to assume the conclusions drawn below are valid for compiled-code implementations too.

3 Performance results

3.1 Test programs

Four test programs were employed.

- **STG**: A STG machine simulator. This 3800 line program parses a Core² file, converts it to STG code, then runs a STG machine simulation. The parser and

² As discussed in [Pey91], Core is a simple functional language used as an intermediate form in the Glasgow Haskell compiler.

Core-to-STG conversions take a lot of heap. Simulation was kept to a minimum so as to hold heap residency high.

- **TreeGrow:** Binary tree traversal. This program was specifically designed to take a lot of heap and have a mean-to-peak residency ratio approaching 100%. It builds a large binary tree which is subsequently traversed a number of times, counting the number of nodes. To make things more interesting, the tree is inverted before counting. The inversion forces a complete copy to be made. Consequently average residency is about 75% of peak.
- **Primes:** Generates an infinite list of primes using a lazy sieve of Erasthones. This program has a low residency, and is extremely lazy.
- **Queens:** All 92 solutions to the 8 queens problem (generated in an inefficient way, so as to give significant run time). Also relatively low residency.

Timings below were obtained by running each program three times under the relevant conditions and averaging. Unless otherwise stated, measurements were obtained with a Sun-4/330 with 32 megabytes of real memory. Results are organised as follows.

Firstly, variation of the following quantities with heap size is shown:

- Garbage collection time.
- Proportion of updates in the old generation.
- Maximum size of the forward reference table.

Next we demonstrate that the gains from generational collection far outweigh the additional mutator cost, especially when heap residency is high, whilst still giving performance as good as semispace collection when the heap is nearly empty. This is done by running the **TreeGrow** and **Queens** programs using semispace and compacting mark-scan (CMS) collectors as well as with the generational scheme.

The generational scheme presented above has one parameter which can be adjusted: the maximum proportion of the heap that the old generation may occupy. In terms of Figure 1, this is the value of **OldMax**. We show how collection time varies with this quantity and thereby justify the choice of 90% used in all other measurements.

3.2 Generational collector performance

All heap sizes quoted are in cells. For the purposes of comparison, the maximum known residencies of the test programs are shown below. Average mutator time is also shown. Variation of mutator time with heap size is very small.

Program	Maximum Residency	Avg Mutator Time
STG	89000	70.3 $\pm$ 0.3
TreeGrow	112000	48.2 $\pm$ 0.2
Primes	20800	329.5 $\pm$ 2.5
Queens	22500	616.0 $\pm$ 8.2

3.2.1 Collection Time vs Heap Size

Collection time is presented as a percentage of average mutator time.

Heap Size	STG	TreeGrow	Primes	Queens
100000	10.9	-	16.3	2.08
120000	7.17	-	14.8	1.87
140000	5.94	9.19	13.7	1.84
160000	5.34	10.1	12.9	1.64
180000	5.01	11.2	12.4	1.50
200000	4.97	7.23	11.5	1.50
240000	5.05	8.26	10.6	1.35
280000	4.43	6.20	9.92	1.21
350000	4.40	6.55	9.04	1.20
400000	4.20	4.37	8.56	1.03
500000	3.59	4.34	7.98	1.01

The high-residency cases, **STG** and **TreeGrow** are encouraging. Given that **STG** has a maximum residency of 89000 cells, it is impressive that the collection time is only 11% of mutator time with a 100000 cell heap. Even small increases in heap size cause this figure to fall rapidly.

The **TreeGrow** program shows similar good collection times even when the heap is only marginally larger than the maximum residency. In this case, collection time falls off jerkily as the heap expands. This curious phenomenon merits further investigation.

The **Queens** program does not exercise the heap very much, hence the low collection times. It is doubtful whether a semispace collector could do much better in this case.

Unfortunately the **Primes** program, which also has a moderate residency, runs against these otherwise hopeful results. This is due to the rather unusual dynamic behaviour of the program. Section 3.2.4 discusses the matter further.

3.2.2 Forward Reference Table Size vs Heap Size

The sizes presented are the maximum observed forward reference table size at minor-collection time.

Heap Size	STG	TreeGrow	Primes	Queens
100000	510	-	4418	2009
120000	802	-	4330	2005
140000	903	37	4210	2010
160000	1113	39	4435	1996
180000	675	37	4384	2013
200000	809	36	4314	1991
240000	1193	37	4339	2000
280000	1624	47	4338	1998
350000	2293	40	4493	1997
400000	2733	48	4387	2016
500000	3393	45	4315	1904

The results are curious, but encouraging. For **TreeGrow**, **Primes** and **Queens**, the table size is essentially independent of heap size. Why could this be? A plausible hypothesis is as follows. As the heap expands, the interval between new-space collections increases. Consequently, it becomes more and more likely that updates overwrite new-space rather than old-space cells. On the other hand, since the mutator runs for longer between minor collections, it has the potential to generate more updates and therefore more old-space updates. Let the term *thunk delay* denote the number of reductions which elapse between the creation of a thunk and the updating of it. If we now assume that the thunk delays have a negative exponential distribution, the two phenomena could cancel each other out.

The STG results suggest that, despite a few kinks, table size in this case is proportional to heap size. Even in this case, that's quite acceptable. From all the measurements above, the maximum table size is 4493 entries. Hence, table size to heap size is

$$\frac{4493}{500000 \times 2} = 0.45\%$$

This is a conservative estimate, assuming all closures (in a variable-closure-size heap) are 2 words long. As mentioned in Section 2.3, it is not a disaster even if the table does overflow from time to time: we simply have to start the minor collection before **FromSpace** is full.

3.2.3 Old Space Updates as a proportion of Total Updates

This title is not quite accurate. What is actually shown is the percentage of all updates which require an entry in the forward reference table. Recall that such an entry need be made only for those old-space updates for which the overwriting cell contains pointers, one or more of which point to the new generation.

Heap Size	STG	TreeGrow	Primes	Queens
100000	1.56	-	31.3	0.876
120000	1.53	-	27.2	0.799
140000	1.40	0.246	24.0	0.787
160000	1.19	0.235	21.5	0.745
180000	1.10	0.208	19.8	0.692
200000	1.14	0.142	17.7	0.687
240000	1.17	0.140	15.0	0.625
280000	1.06	0.097	13.1	0.594
350000	0.89	0.086	10.6	0.562
400000	0.90	0.068	9.31	0.526
500000	0.81	0.044	7.69	0.492

Once again, the **Primes** result spoils otherwise encouraging news. We look at this in the next section. For the others, it is instructive to see that the maximum proportion of updates entered in the forward reference table is about 1.6%, and in many cases dramatically lower.

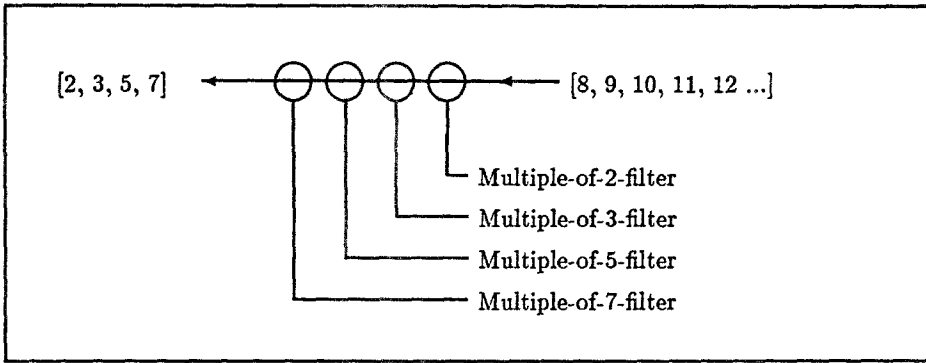


Figure 2: The infinite sieve of Eratosthenes, immediately after 7 has been discovered prime

Although not listed here, the total number of updates in old-space, whether or not the overwriting cell contains pointers, was also recorded. This indicates that about two-thirds of all old-space updates had to be entered into the forward reference table. Ignoring **Primes**, this means that at worst 97.6% of all updates are *not* in old-space. This is important, because we can detect a non old-space update simply by comparing the updated address with the old-space-end pointer, **OldEnd**. Provided both values are in registers, this imposes a tiny overhead on this 97.6% of all updates.

3.2.4 Effect of dynamic behaviour on performance

Many systems rely on statistical properties of the problem in hand to get good performance in the majority of cases. Freak cases that do not “play along” cause severe performance degradation. For example, virtual memory systems and caches lose effectiveness if the programs being run do not exhibit enough locality. Similarly, if everyone in the country decides to telephone the Prime Minister at the same time, the phone system will virtually collapse.

Unfortunately for us, a generational collector is just such a system. For it to work well, it is necessary for old-space updates to only be a very small proportion of all updates. We observe that the **Primes** program has a large number of old-space updates and hence an inordinately large collection time. An important question is why there should be so many old-space updates.

Figure 2 shows the sieve in progress. The printing mechanism “pushes” a barrage of filters along the candidate list of numbers [8, 9, 10, 11, 12 ...]. Candidates which get through are prime and join the output list [2, 3, 5, 7]. Each new prime also attaches a “is-not-a-multiple-of-me” filter to the barrage. In the example, the next successful candidate, 11, attaches a fifth filter.

After a while, many prime numbers will have been produced, so there is a correspondingly large collection of filters. To get the next prime, the printing mechanism kicks the leftmost filter, which in turn kicks its neighbour. Demand is propagated through all filters, right up to the candidate list. Now, clearly the first few filters, for multiples of 2, 3, 5 and 7, filter out the majority of candidates, so it may be a long time before a hopeful makes it back to the leftmost filters. These have all been awaiting update for a

long time, and may well have been transported to old-generation by a minor collection. Consequently, a heavy burden is imposed on the collector.

So there is at least one program to which generational collection does not respond well. Is that a good reason to abandon this approach? We believe not.

- Although collector performance in this case is not as good as we would like, it is not bad: perhaps two or three times the cost of semispace collection in a moderate-sized heap.
- The other three programs examined respond extremely well to generational collection. In particular, it is significant that the STG program, of considerable complexity (3800 lines of code), works well. It seems a pity to throw away this large average-case performance gain.
- One hopes that most programs update most of their thunks quickly, so average-case behaviour is almost always exhibited.

Examining the distribution of thunk delays (defined in Section 3.2.2) provides more information than merely looking at the average value. It would appear that the effectiveness of generational collection in an environment with updates depends centrally on this distribution. Further work in this area could be very useful.

3.3 Performance relative to non-generational collectors

3.3.1 What measures are of significance?

In this section, we hope to answer the following questions:

- Does generational collection really give significant performance benefits?
- To what extent is the mutator impaired by the need to record old-space updates?

What is the best way to measure collector efficiency? First, observe that for most types of collectors, efficiency can be improved to some extent simply by making the heap larger. Observe also that in many cases, the utility of functional programs is limited not by collection time, but by the size of heap needed to keep collection time down to a reasonable level.

Other workers [San91] have measured GC efficiency in terms of the rate at which free heap is reclaimed. We suggest a more natural measure is to directly relate how heap size and overall run time are related. Results below are phrased in this way.

Two programs, **TreeGrow** and **Queens** were selected as well-behaved representatives with high and low residencies respectively. They were run with a range of heap sizes, using a semispace collector, a compacting mark-scan collector, and the generational collector. In the latter case, old-space maximum size was set to 90% of overall heap size, as for the measurements above.

3.3.2 The TreeGrow program

Average mutator time

Collector	Mutator Time
Semispace	50.0 $\pm$ 1.6
CMS	48.0 $\pm$ 2.6
Generational	48.2 $\pm$ 0.4

Mutator time with each different collector was averaged over all runs. Measurement noise seems to swamp any detectable variation in the generational case.

Overall run time

Heap Size	Semispace	CMS	Generational
130000	(heap overflow)	112.3	57.1
140000	(heap overflow)	97.6	52.9
150000	(heap overflow)	88.5	53.3
160000	(heap overflow)	83.5	53.6
170000	(heap overflow)	79.3	53.8
180000	(heap overflow)	76.3	54.4
190000	(heap overflow)	73.2	51.8
200000	(heap overflow)	71.0	52.1
250000	113.1	-	-
300000	77.6	62.0	51.5
400000	63.6	59.9	50.5
500000	59.4	57.0	50.5
600000	57.6	56.5	50.7
700000	55.0	56.6	50.5
800000	56.2	56.0	50.7
900000	53.2	55.2	50.5
1000000	52.2	57.0	50.4

The generational collector outperforms the other two under at all heap sizes, spectacularly so when the heap is relatively small. Recall that maximum residency of this program is about 112000 cells. It is noteworthy that generational GC time does not improve much once heap size exceeds about 140000 cells. This suggests that residency can get up to about 80% with practically no performance penalty. The second significant observation is that the generational collector outdoes the semispace collector even when the heap is very large, the best-case for semispace collection.

Required heap sizes

Finally we phrase the question the other way round, and ask how much heap is needed to get overall run time down to a given level. Heap sizes are approximate.

Overall time	Semispace	CMS	Generational
65	390000	270000	≤ 125000
60	490000	400000	125000
55	700000	900000	140000

These figures speak for themselves.

3.3.3 The Queens program

Peak residency of 22500 cells gives extremely low residency at all heap sizes used (22.5% down to 2.25%), so the semispace collector can be expected to do well.

Average mutator time

Collector	Mutator Time
Semispace	621.5 ± 21.3
CMS	614.1 ± 3.4
Generational	617.8 ± 11.8

As before, measurement noise predominates.

Garbage collection time

Because heap occupancy is so small, garbage collection time is similarly small. Presenting overall run times is made rather meaningless by the measurement noise for mutator time, so only the garbage collection time is shown. Observe in many cases how it declines to less than 1% of mutator time.

Heap Size	Semispace	CMS	Generational
100000	38.38	76.94	12.85
200000	17.45	65.30	9.65
300000	11.26	61.52	7.53
400000	7.97	60.35	6.58
500000	7.03	58.23	6.54
600000	5.59	58.19	6.42
700000	5.06	57.27	5.86
800000	4.32	56.74	5.30
900000	3.65	56.20	4.99
1000000	3.07	56.55	4.89

As expected, compacting mark-scan collection does badly because of the low residency. The semispace collector draws ahead of the generational collector as peak occupancy sinks beneath about 4% (corresponding to 562500 cells), but collection time in both cases is so small that this makes little difference.

The conclusion to be drawn here is that the generational collector outperforms the other two once heap residency is above a few percent, and does spectacularly well in the important cases where residency approaches 100%. Also significant is the fact that no significant impairment of mutator performance can be detected in the generational case.

3.4 Old space size as a proportion of total heap size

At any given heap size, there is another parameter to adjust: what proportion of the heap the old-space may occupy. In terms of Figure 1, this is the position of **OldMax**. Since this value defines the absolute maximum heap occupancy allowed, we would like to get it as high as possible consistent with reasonable performance. However, both a very high and very low setting degrade efficiency, since:

- As **OldMax** decreases, the frequency of major collections increases. Major collections are expensive.
- As **OldMax** increases, the frequency of minor collections increases, so the interval between them decreases. This erodes the effectiveness of generational collection, since it decreases the average age of cells moved to the old generation.

A secondary consideration is that minor collections, although cheap, are not free, especially if there are a lot of roots around (a large G-machine stack or a lot of old-space updates).

This suggests there is some mid-range setting which gives optimal efficiency. Clearly, this depends both on the dynamic properties of the particular program being run, and on the relative speeds of the compacting mark-scan and semispace collectors. We can only hope the former effect does not make much difference, select a "representative" test program and conduct some trial runs to arrive at a value.

Figures below are for the **STG** program³. Absolute heap size is held constant at 200000 cells whilst the proportion allocated to the old-generation is varied from 45% to 97.5%. The program has a maximum residency of 89000 cells (44.5% of the heap).

Old-space size	Total GC time
45%	6.22
50%	5.14
55%	4.98
60%	4.78
65%	4.66
70%	4.28
75%	4.39
80%	3.98
85%	4.21
90%	4.31
95%	4.51
97.5%	4.52

³ Run on a Sun-4c/60, 8 megabytes real memory.

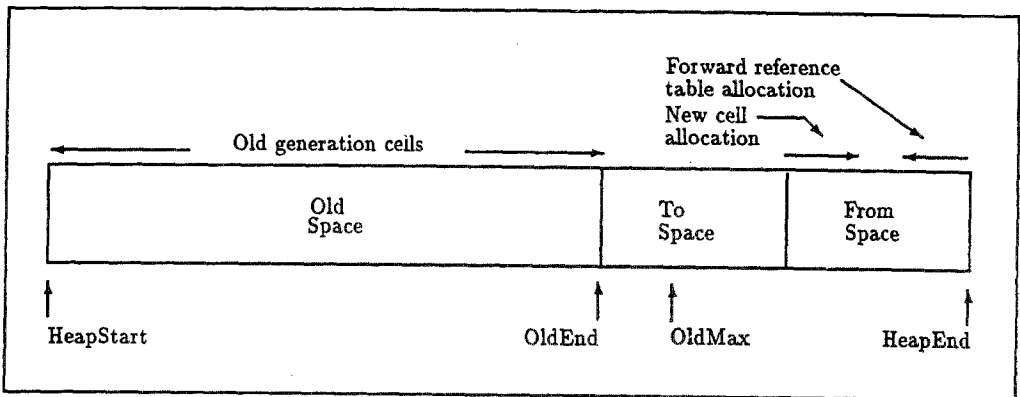


Figure 3: Putting the forward reference table in the heap

Despite a small wiggle, this suggests that any size from 70% to 90% will do, and even going up to 97.5% does little harm. This is good, since it means heap utilisation approaching 100% is quite feasible.

4 Optimisations to the basic scheme

Once it is established that generational collection does not impose an excessive burden on the mutator, a large design space opens up. In this section, some ideas which may improve performance are discussed. None of them have been implemented, though, due to lack of time.

4.1 Putting the forward reference table in the heap

Having a forward reference table which may overflow, or be underutilised, independantly of what's going in the heap, is a nuisance. An obvious place to put the table is right at the top of the heap, as shown in Figure 3. Minor collections now occur when new-cell allocation crashes into the top of the table. This is the usual “two stacks in one array” trick in disguise. The advantage is it gets rid of a separate table. On the other hand, it does make minor collections a little more frequent, with the associated costs discussed in Section 3.4. Hopefully, if the program being run is well-behaved, it will not generate many old-space updates, so the diminution in `FromSpace` size is minimal.

Another, more elegant solution is available for systems which can update using indirection nodes. All old-space updates are done with indirections. Each indirection node contains a field which is used to point to another indirection node, so, as old-space updates occur, a linked list of indirection nodes is built up. This list is, in effect, the forward reference table.

4.2 Making FromSpace larger than ToSpace

The reason we make **ToSpace** the same size as **FromSpace** is to cope with the worst case minor-collection wherein all **FromSpace** cells are live. In practice this rarely happens. The collector is at its most effective when the number of live **FromSpace** cells is minimised. Observations show that typically less than 10% of **FromSpace** cells are live at minor collection time, with many programs getting below 3%. Values above 30% are rare. Andrew Appel quotes a corresponding figure of 1.87% in [App92].

Given that increasing **FromSpace** size increases collector efficiency, it seems a great pity to waste most of **ToSpace**. The Big Idea here is to monitor **FromSpace** residency, and dynamically adjust the relative **ToSpace/FromSpace** ratio accordingly. It appears that **FromSpace** residency changes relatively slowly, so guessing the required next **ToSpace** size, as, say, 50% more than the previous **FromSpace** residency keeps things safe in the face of **FromSpace** residency increases of up to 50% between adjacent minor collections.

Since we clearly cannot *guarantee* anything about residency changes, there must be a way to deal with the case where the live cells of **FromSpace** do not fit into the allocated **ToSpace**. We observe (as in [San91]) that the heap is always in a consistent state during semispace collection. Consequently, if **ToSpace** overflows during a minor collection, the minor collection can be abandoned and the major collector called instead. Given that major collections are expensive, this had better happen only extremely rarely. The key question is how to make a good guess of how big to make **ToSpace** after each minor collection. All manner of clever schemes come to mind: it will be interesting to see how well they perform.

4.3 Multiple minor collections before merging

All the previous variants suffer from an annoying deficiency in that the old generation grows ever larger at every minor collection. This forces the occasional major collection even if none of the live cells collected during minor collections is really destined to become “genuine” old generation data.

It is easy to modify the collector to do multiple minor collections before merging the result of a minor collection onto the old generation. An interesting question is exactly when such a merge should occur. We might stipulate that this happen whenever the semispace residency exceeds, say, 20%. For a program with low residency, this means the system acts perpetually like a semispace collector. Alternatively, the merging could take place every n 'th minor collection.

Quite how such a scheme affects performance is unknown. Since the existing implementation works well in the majority of cases, perhaps development should concentrate on improving the worst case, as typified by the **Primes** program.

A word of warning about these complicated schemes is in order. The more parameters which can be twiddled, the smaller is the chance that we will ever arrive at an optimal setting, or even that one setting is optimal for all programs. Building mathematical models of collector-mutator performance may help, but at the end of the day it is often down to time consuming experimentation.

4.4 Compiled code implementation

A popular way to implement fast case-analysis in compiled implementations is by having the tag field of each cell point to a so-called “info table”. For every possible action, the info table contains a pointer to the code that performs the action for this type of cell. Further details may be found in [Joh87] and [Pey91].

In [San91], the details of doing both semispace and compacting mark-scan collection using info tables are presented. The only remaining problem is how to create entries in the forward reference table.

When a cell is updated, we need to decide whether to put its address in the table. In the crudest approach, this involves a comparison of the address being updated with **OldEnd**, followed if necessary by a call to a routine **EnterIntoFPTable** which enters this address into the table. In the following C code, the address being updated is in **UpdAddr**. The code then is:

```
if ( UpdAddr <= OldEnd ) EnterIntoFPTable ();
```

Statistics from Section 3 show the vast majority of updates fail the test. Assuming **UpdAddr** and **OldEnd** are in registers, as explained in [Pey91], the overhead for most updates could be as low as two or three instructions.

A more refined approach only inserts an old-space updated cell into the forward reference table if it contains pointers and one or more of these points to new-space. What we need here is to invent a new method, which:

- Does nothing if the cell contains no pointers.
- Does nothing if the cell contains pointers, but they all point to old-space.
- Otherwise, adds the address of the cell to the forward reference table.

To achieve this, it is necessary to allocate a new info table slot for this action, and write code to perform it for every kind of cell. Since these pieces of code “know” the layout of the cells they operate on, this is a fairly cheap operation. Supposing that **EnterIntoFPTableDISP** is the displacement for this method in info tables, our code might now look like this (neglecting typecasting):

```
if ( UpdAddr <= OldEnd )
    (* ((*UpdAddr) + EnterIntoFPTableDISP)) ();
```

In English, this means: “if **UpdAddr** is in **OldSpace**, call the **EnterIntoFPTable** method for the cell pointed to by **UpdAddr**”.

5 Conclusions

This paper provides strong evidence that generational garbage collection is viable for lazy reduction systems. The small added mutator costs are greatly outweighed by better garbage collector performance. We showed how the central question of recording old-space updates can be dealt with at very little space and time cost.

It is also, unfortunately, apparent that generational collection makes certain assumptions about statistical-level dynamic behaviour of programs. The primary factor here is the distribution of delays between the creation of a thunk and its updating. It may be that these have a negative exponential distribution. Generational collection works well when most such delays are very small. In the few cases where these assumptions do not hold, performance will not be as favourable. However, there is a strong argument to be made that for the vast majority of programs these assumptions are valid. Significantly, a 3800 line program runs very well with this collector.

Finally, some modifications were suggested. Whether or not these are a good idea awaits further work.

6 Acknowledgements

A special thank-you to David Rushall, who implemented large sections of our G-machine interpreter *LazySu* on which this work is based. Dave also wrote much of the STG machine simulator used as a benchmark, and spent many hours debugging and testing.

Financial support was provided by the Science and Engineering Research Council.

Bibliography

- [App92] Andrew W. Appel. *Compiling with Continuations*. Cambridge University Press, 1992.
- [Aug84] L. Augustsson. A compiler for lazy ML. In *Proceedings of the 1984 ACM Symposium on Lisp and Functional Programming*, pages 218–227, Austin, Texas, August 1984.
- [Che70] C. J. Cheney. A nonrecursive list compacting algorithm. *Communications of the ACM*, 13:677–678, November 1970.
- [Joh87] T. Johnsson. *Compiling Lazy Functional Languages*. PhD thesis, Chalmers Tekniska Högskola, Göteborg, Sweden, 1987.
- [Jon79] H. B. M. Jonkers. A fast garbage compaction algorithm. *Information Processing Letters*, 9:26–30, July 1979.
- [Pey87] S.L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice-Hall International Series in Computer Science. Prentice-Hall International (UK) Ltd, London, 1987.
- [Pey91] S.L. Peyton Jones. The spineless tagless g-machine: second attempt. Publication forthcoming, 1991.
- [San91] Patrick M. Sansom. Dual-mode garbage collection. In *Proceedings of the 1991 Glasgow Functional Programming Workshop*, 1991.